

Java Programming Assignment

Topic: Object-Oriented Programming (Polymorphism & Method Overriding)

Question 2: Vehicle Rental System

Scenario

A vehicle rental company rents different types of vehicles to customers. The company currently provides:

- Bike
- Car
- Truck

The rental charge depends on the type of vehicle. The company has defined the following daily rental charges:

Bike → ₹500 per day
Car → ₹1,000 per day
Truck → ₹2,000 per day

When a customer rents a vehicle, the system should calculate the total rental amount based on:

Total Rent = Daily Rent × Number of Days

The system should display the vehicle type, number of rental days, and total rent.

Example

A customer rents a Car for 3 days.

Daily Rent : ₹1,000
Number of Days: 3
Total Rent : ₹3,000

Test Cases

TC	Vehicle	Days	Expected Rent
TC01	Bike	2	₹1,000
TC02	Car	3	₹3,000
TC03	Truck	5	₹10,000
TC04	Bike	10	₹5,000
TC05	Car	7	₹7,000

Student Task

Develop the Java application according to the above requirements.

Additional Requirement: The company may add new vehicle types in the future. The design should allow new vehicle types to be added without changing the existing rental calculation logic.

Hint: Create a base class `Vehicle` with a method `calculateRent()`, and override it in child classes.

Question 3: Bank Account Management

Scenario

A bank wants to develop a simple Account Management System. The bank provides two types of accounts: **Savings Account** and **Current Account**. Every account has an Account Number, Account Holder Name, and Balance.

However, the withdrawal rules are different.

- **Savings Account Rule:** A customer can withdraw money only if the requested amount is not greater than ₹20,000 AND not greater than the available balance.
- **Current Account Rule:** A customer can withdraw money only if the requested amount is not greater than ₹50,000 AND not greater than the available balance.

If the withdrawal is successful, the amount should be deducted from the balance. If it is not allowed, the system should display an appropriate message.

Example

```
Savings Account:
Balance : ₹50,000
Withdrawal : ₹10,000

Result : Withdrawal Successful
Remaining : ₹40,000
```

Test Cases

TC	Account Type	Balance	Withdrawal	Expected Result
TC01	Savings	₹50,000	₹10,000	Success
TC02	Savings	₹50,000	₹25,000	Failed
TC03	Current	₹50,000	₹40,000	Success
TC04	Current	₹50,000	₹60,000	Failed
TC05	Savings	₹5,000	₹10,000	Failed

Student Task

Develop the Java application according to the given banking rules.

Question 4: Online Payment System

Scenario

An online shopping company wants to develop a Payment Processing System. Customers can make payments using: **Credit Card, UPI, or Net Banking.**

Although all three are used for making payments, the payment processing message should be different:

Credit Card → Payment of ₹5,000 processed through Credit Card
UPI → Payment of ₹5,000 processed through UPI
Net Banking → Payment of ₹5,000 processed through Net Banking

Business Rules:

- Payment amount must be greater than 0.
- The system should display the payment method and the amount being processed.
- Invalid amounts should not be processed.

Test Cases

TC	Payment Method	Amount	Expected Result
TC01	Credit Card	₹5,000	Payment processed
TC02	UPI	₹2,000	Payment processed
TC03	Net Banking	₹10,000	Payment processed
TC04	UPI	₹0	Invalid amount
TC05	Credit Card	₹-500	Invalid amount

Student Task

Develop the Java application based on the above requirements.

Question 5: Shape Area Calculator



Scenario

A drawing application needs to calculate the area of different shapes. The application supports **Circle**, **Rectangle**, and **Triangle**.

The area should be calculated using the following formulas:

```
Circle: Area =  $\pi \times \text{radius} \times \text{radius}$ 
Rectangle: Area = length  $\times$  width
Triangle: Area =  $0.5 \times \text{base} \times \text{height}$ 
```

The application should accept a shape and its required dimensions and display the calculated area.

Example

```
For a rectangle:
Length = 10
Width = 5

Area = 50
```

Test Cases

TC	Shape	Input	Expected Area
TC01	Circle	Radius = 7	153.94 approx.
TC02	Rectangle	10 $\times$ 5	50
TC03	Triangle	Base = 10, Height = 6	30
TC04	Rectangle	20 $\times$ 4	80
TC05	Triangle	Base = 8, Height = 5	20

Student Task

Develop a Java application that can calculate the area of all supported shapes.

Additional Requirement: The design should make it easy to introduce another shape in the future without modifying the core system.

Hint: Consider using an Abstract Class `Shape` with an abstract `calculateArea()` method.

Question 6: Food Delivery Charges



Scenario

A food delivery company charges customers differently depending on the delivery method. The available delivery methods are **Bike**, **Car**, and **Drone**.

Bike → ₹30 base charge + ₹10 per km
Car → ₹50 base charge + ₹15 per km
Drone → ₹100 base charge + ₹25 per km

The total delivery charge is calculated as:

Total Charge = Base Charge + (Per KM Charge × Distance)

Example

A Bike delivery travels 5 km:

Base Charge = ₹30

Distance = 5 km

Per KM = ₹10

Total = 30 + (10 × 5) = ₹80

Test Cases

TC	Delivery Type	Distance	Expected Charge
TC01	Bike	5 km	₹80
TC02	Car	10 km	₹200
TC03	Drone	4 km	₹200
TC04	Bike	10 km	₹130
TC05	Car	20 km	₹350

Student Task

Develop the application according to the business requirements.

Question 7: Hospital Treatment System



Scenario

A hospital wants to develop a system for handling different types of doctors. The hospital has a **Cardiologist**, **Neurologist**, and **Orthopedic Doctor**.

Every doctor treats patients, but the treatment description depends on the doctor's specialization:

Cardiologist → Treats heart-related problems
Neurologist → Treats nervous-system-related problems
Orthopedic Doctor → Treats bone and joint problems

The hospital's reception system should be able to work with any doctor registered in the system (Polymorphic behavior).

Test Cases

TC	Doctor Type	Expected Treatment
TC01	Cardiologist	Heart-related treatment
TC02	Neurologist	Nervous-system treatment
TC03	Orthopedic	Bone/joint treatment

Student Task

Develop the application ensuring the reception system utilizes runtime polymorphism to call the correct treatment method based on the doctor provided.

Hint: Create an Interface Doctor with a method treatPatient(). Let specific doctors implement this interface.